

Bradian Database Systems approved primary horizontal logo

BRADIAN DATABASE SYSTEMS

Master Product Architecture, Software Requirements Specification, Security Architecture and Implementation Programme

Product: Bradian Database Systems
Developer: Vitalworks Limited
Document: BDS-MASTER-001, Revision 1.1
Version: 1.1 — Expanded Target Architecture
Date: 12 November 2026
Classification: Product Engineering / Architecture Baseline
Status: Proposed requirements; capabilities described here are not claims of an implemented or certified product.

Product mission. Bradian is to become an independent, proprietary, general-purpose relational and analytical database management system with native OLTP, OLAP and hybrid transactional/analytical processing (HTAP), secure built-in horizontal scaling, federated installations, and a professional browser-based management environment. It must run without a mandatory third-party database engine, while allowing reviewed implementation libraries and optional interoperability connectors.

Revision 1.1 change control

Version 1.1 preserves every v1.0 core subsystem and adds normative detail for distributed commit ordering, crash-safe storage, secure query execution, cluster automation, online schema evolution, migration, durable jobs and cluster-consistent recovery. It also supplies original implementation diagrams, testable protocol choices and explicit engineering steps. All diagrams are **target designs** rather than depictions of running software. Where a decision is still open, an architecture decision record (ADR) and acceptance tests are mandatory before durable interfaces are frozen.

Document control and source basis

This master document expands the previously supplied *DER Analytics Platform — Comprehensive Product, Storage, Compute and Distributed Analytics Architecture*, version 1.0, dated 11 November 2026. That baseline provides the original proprietary columnar engine, vectorized execution, cost-based optimization, incremental data products, distributed site analytics, durability and performance goals. The independent relational DBMS, portable database service, HTAP commit model, built-in cluster routing, browser Studio and expanded security programme are **new Bradian requirements**, not features already specified in detail by the baseline. The approved Bradian horizontal falcon/database/analytics logo is

retained as the visual identity, without redesign. All implementation claims must be established by tests and evidence.

Normative language: MUST denotes a mandatory product requirement; SHOULD denotes an expected default requiring a recorded exception; MAY denotes a scoped extension. Requirements carry stable identifiers. A feature is *not complete* until its correctness, authorization, fault and operational tests pass.

1. Executive product vision and boundaries

1.1 Positioning

Bradian is a database system, not merely a reporting tool. An application must be able to create a database, define related tables, enforce constraints, insert and update rows inside transactions, recover after a crash, connect through a documented protocol, and run intensive analytical SQL without exporting data to a different product. Its brand architecture separates Bradian Core (database kernel), Transaction Engine, Analytics Engine, Hybrid Query Engine, Cluster Fabric, Stream, Connect and Bradian Studio. These are logical product subsystems, not an instruction to create isolated processes for every responsibility.

The original DER focus remains a competitive strength rather than a design constraint. Bradian must support hospital and DHIS2/DER installations through adapters, but its core metadata, SQL, permissions, storage and planning are generic. Finance, commerce, government, manufacturing and other users must be able to deploy Bradian without importing health-specific schema or runtimes.

1.2 Non-goals and realistic commitments

Bradian does not claim universal superiority to ClickHouse, Doris, PostgreSQL, SQL Server or any other system. Success must be benchmarked on disclosed hardware, datasets, concurrency, correctness, consistency and recovery behaviour. Nor must Bradian pretend that a geographically disconnected installation can simultaneously provide local unrestricted writes and global synchronous serializability. The product exposes explicit strong-cluster and autonomous-federation modes, with different conflict and availability contracts.

The platform must own its persistence format, transaction semantics, catalog, distributed coordination, query planning and execution control. It may use audited libraries for cryptography, compression, networking and memory primitives; proprietary means independently controlled database architecture, not inventing vulnerable custom cryptographic algorithms. Licensing, IP ownership, patents and third-party notices require counsel review before commercial release.

1.3 Main user groups and quality attributes

Database administrators manage creation, tuning, security, backup and recovery. Developers use SQL, drivers, migrations and observability. Data engineers ingest, transform and govern data; analysts query and publish data products; platform operators scale clusters and diagnose failures. Auditors inspect policy, access and immutable evidence. The principal quality attributes are correctness, durability, confidentiality, availability, performance, interoperability, maintainability and portability, with security and safety overriding speed optimizations.

2. Logical architecture and component relationships

2.1 End-to-end execution structure

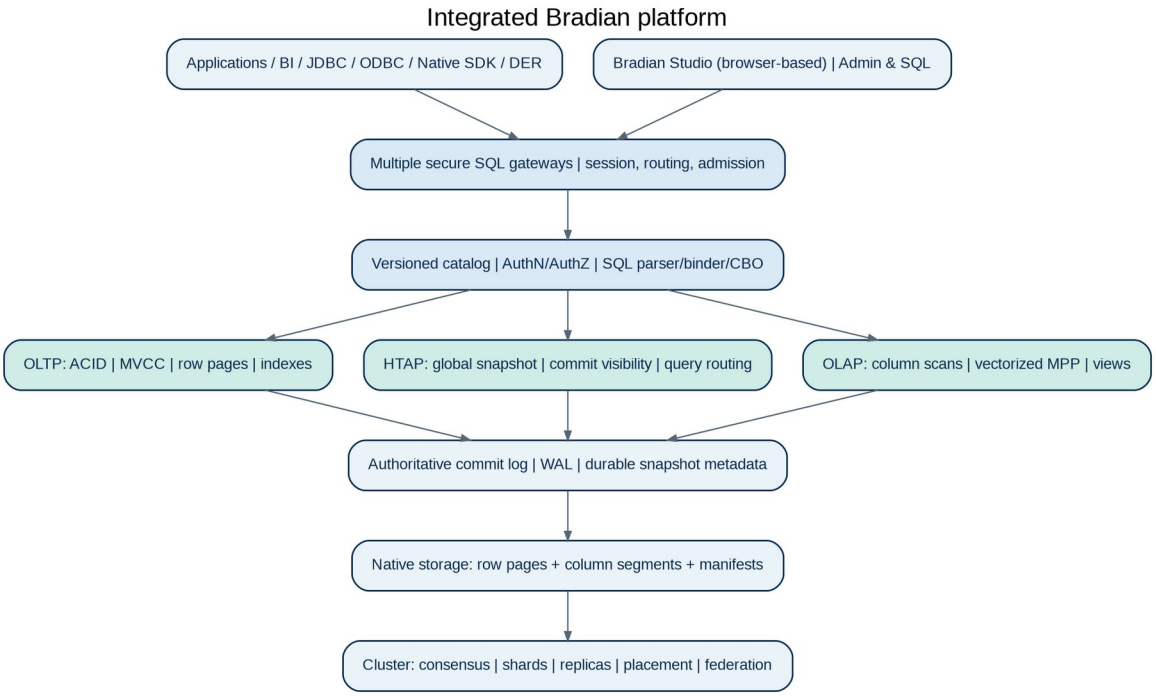


Figure 01_platform: Integrated Bradian platform

Figure: Integrated Bradian platform. Reference design; paths and protocols require implementation and validation.

```
Applications / JDBC / ODBC / Native SDK / REST / Bradian Studio
                  |
        Bradian Gateways + Secure Session Router
                  |
      Authentication / Authorization / SQL Front End
                  |
       Unified Catalog + Parser + Binder + Optimizer
                  |
      +-----------------+-----------------------+
      | Transaction     | Hybrid Query          | Analytical
      | Engine (OLTP)   | Coordinator (HTAP)    | Engine (OLAP)
      +-----------------+-----------------------+
                  |
         Transaction Commit Log + Snapshot Service
                  |
         Row Store  <=>  Committed Column Projections
                  |
    Cluster Membership / Shards / Replicas / Federated Links
                  |
      Disk / Memory / Network / OS Portability Layer
```

Cross-cutting: key management, policy enforcement, auditing, resource governor, backup/PITR, telemetry, admission control.

This architecture deliberately avoids two separately authoritative databases. The row-oriented transactional representation and committed transaction log establish authoritative state; columnar segments and materialized views are versioned derivatives. The hybrid coordinator chooses a suitable access path and must never combine incompatible snapshots. A single-node deployment may host every subsystem within one service; larger installations can assign explicit gateway, storage, transaction and analytical worker roles.

2.2 Bradian Core and platform abstraction

The core manages type definitions, value lifetimes, memory allocation, error contracts, scheduling and durable object identities. The portability layer isolates OS file APIs, directory durability, atomic rename limitations, clocks, random sources, sockets, TLS bindings, threads and system-service integration. Storage files use fixed endian and alignment rules rather than writing raw compiler structures. Bradian must build and pass filesystem crash tests independently on Windows and supported Unix-like systems.

A common logical plan and type system serves every deployment profile. CPU-specific SIMD acceleration is enabled only after detection and must have equivalent portable execution. The server must be operable with small standalone hardware as well as high-volume multi-node hardware; a mandatory Kubernetes, cloud account, object store or external load balancer would violate the standalone requirement.

2.3 Product modules and contracts

The transaction engine exposes a versioned commit and snapshot interface; the column builder consumes committed changes and publishes watermarks; the SQL front end binds names and types through the generic catalog; the optimizer requests cardinality and storage statistics; the executor consumes typed batches; and the gateway enforces authenticated session and query admission. Each cross-module contract must define wire format, errors, cancellation, version negotiation, access checks, telemetry and backward compatibility. The public interfaces must be documented separately from unstable private internals.

3. Relational data model and SQL compatibility

3.1 Databases, schemas, tables and object catalog

Bradian MUST support multiple databases and schemas, typed columns, primary and secondary indexes, ordinary views, sequences, partitions, constraints and governed ownership. The catalog maps stable object IDs to names and versions, and records dependencies before allowing a destructive schema change. The first complete SQL target should be a published, testable subset of SQL with CREATE, ALTER, DROP, INSERT, UPDATE, DELETE, MERGE or a clearly defined UPSERT, SELECT, joins, CTEs, grouping and ordering. Unsupported syntax must fail clearly and not silently discard semantics.

Identifiers require documented case folding, Unicode and quotation rules. Schema changes must preserve snapshot safety; long-running queries cannot suddenly interpret a column using an incompatible type. Online schema migration should stage new structures, backfill under controlled budgets, validate constraints and switch catalog versions atomically when possible. Dependencies among views, indexes, foreign keys and materializations must be inspectable in Studio.

3.2 Relational constraints and referential actions

Primary keys, unique keys, composite keys, NOT NULL, CHECK, defaults, identity columns and foreign keys form the integrity model. Foreign-key checks must be integrated into the same transaction as row mutations, including cascading changes, RESTRICT and SET NULL according to a precise published

subset. Constraints cannot be approximated in a background analytics projection: an invalid transaction must be rejected before acknowledgement. Deferred constraint support, if offered, must be checked at commit.

CREATE TABLE and relationship-design screens are simply two representations of the same catalog operation. The UI must not maintain a separate metadata schema from SQL. Index-building and constraint-validation jobs report progress, old/new schema visibility and impact on concurrent write traffic. Validation failures must identify the affected constraint without exposing confidential values to unauthorized clients.

3.3 Type system and functional SQL

The mandatory core includes signed and unsigned integers with explicit overflow rules, exact decimals, IEEE floating point, booleans, UTF-8 strings, binary values, date/time/timestamp with timezone semantics, UUIDs, NULL and structured JSON. Decimal rounding, collation, Unicode comparison and timestamp normalization must be deterministic and documented. Arrays, geospatial types, vector embeddings, full-text and extension-defined types can follow as independently tested capabilities.

Bradian SQL must support parameter binding, prepared statements, scalar expressions, CASE, NULL semantics, subqueries, aggregate functions and window functions appropriate to the defined conformance profile. User-defined functions must execute in a restricted runtime with capability controls; SQL must not be able to invoke arbitrary OS commands or read unrelated files. SQL conformance tests and differential tests against reference systems should be part of release qualification.

3.4 Illustrative relational transaction

```
CREATE TABLE customers (
  customer_id BIGINT PRIMARY KEY,
  display_name VARCHAR(200) NOT NULL
);
CREATE TABLE invoices (
  invoice_id BIGINT PRIMARY KEY,
  customer_id BIGINT NOT NULL REFERENCES customers(customer_id),
  amount DECIMAL(18,2) NOT NULL CHECK (amount >= 0)
);
BEGIN;
INSERT INTO customers(customer_id, display_name) VALUES (1001, 'Example Customer');
INSERT INTO invoices(invoice_id, customer_id, amount) VALUES (9001, 1001, 125000.00);
COMMIT;
```

This script expresses the intended SQL surface. It is not evidence that an existing Bradian binary accepts these statements. A release gate must prove atomicity, constraint enforcement, crash durability, isolation and cross-platform recovery for these operations.

4. OLTP transaction engine and concurrency

4.1 Transaction lifecycle and ACID

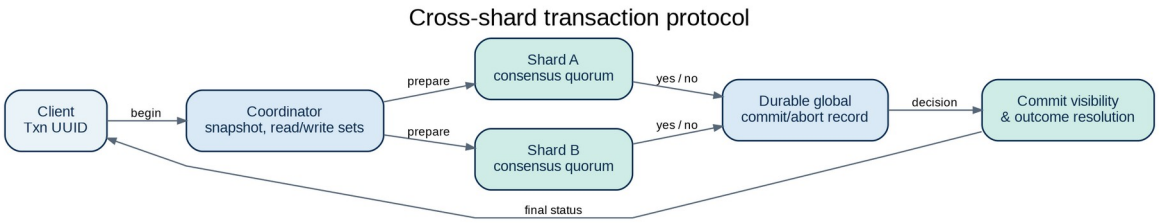


Figure 02_transaction: Cross-shard transaction protocol

Figure: Cross-shard transaction protocol. Reference design; paths and protocols require implementation and validation.

The transaction manager assigns transaction identity, captures snapshot rules, tracks read/write sets, validates constraints, coordinates locks or MVCC conflicts, durably commits and publishes changes to downstream projections. BEGIN, COMMIT, ROLLBACK and savepoints must behave predictably, including failure within a multi-statement transaction. Commit acknowledgment may be issued only after the configured durability contract is met. Asynchronous replication must never be mislabeled as synchronous durability.

Atomicity means that a crash, exception or authorization failure cannot leave half an invoice or partially updated related rows visible. Consistency encompasses declarative constraints and documented invariants. Isolation must be defined by level and validated with adversarial schedules; durability depends on WAL ordering, OS flush semantics, checksums and tested recovery. Bradian must publish specific guarantees rather than rely on the acronym ACID alone.

4.2 MVCC, locking and isolation levels

A version chain or equivalent MVCC structure lets readers observe an appropriate committed snapshot while writers modify rows concurrently. Read Committed and Repeatable Read should be implemented and verified before Serializable is advertised. Serializable requires a defensible anomaly-prevention method such as serializable snapshot isolation with dangerous-structure detection or a proven locking protocol; simple snapshot isolation alone does not prevent all serialization anomalies.

The engine must provide row-level conflict management, deadlock detection or prevention, bounded lock waits, optimistic retry signals, transaction deadlines and session cancellation. Version garbage collection is constrained by active snapshots and replicas so cleanup cannot remove versions still required by readers, backups or replication. Tests must cover lost updates, write skew, phantom reads, uniqueness races, cascading deletes and concurrent DDL.

4.3 Row-oriented storage and primary indexes

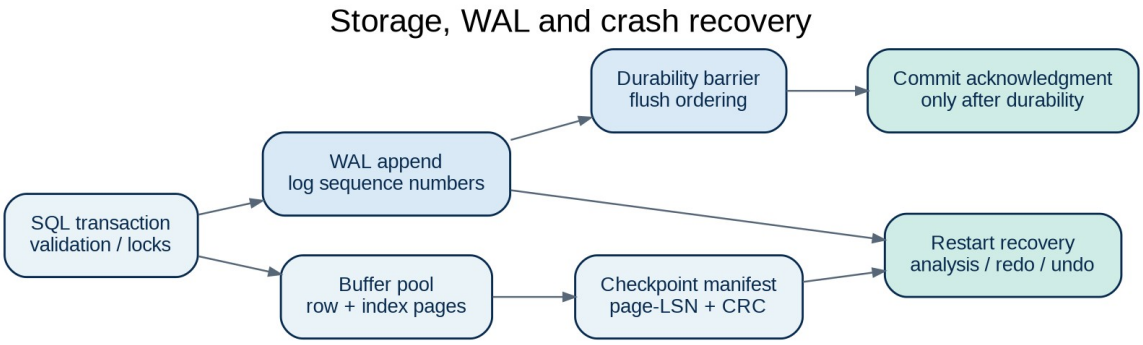


Figure 03_storage: Storage, WAL and crash recovery

Figure: Storage, WAL and crash recovery. Reference design; paths and protocols require implementation and validation.

The transaction store should use an independently defined portable page format, with page IDs, checksums, version metadata, free-space management and recovery-safe allocation. A B+tree or comparable range-capable index serves primary-key and selective transactional lookups; secondary indexes are kept transactionally consistent with row mutations. Page cache, buffer eviction, hot-key handling, write batching and checkpointing are controlled by resource budgets so high ingestion does not cause unbounded memory consumption.

Row pages, index pages and WAL require careful ordering under crash and power-loss faults. Recovery must distinguish acknowledged transactions from incomplete transactions, redo committed updates where needed and undo or discard incomplete changes. A page scrubber, checksum verification and quarantined corruption mode protect against silently returning invalid data.

4.4 Connection and session semantics

Each connection has an authenticated identity, database context, transaction state, prepared statements, query limits and cancellation controls. The session service handles connection limits, idle timeouts, keepalives, credential rotation and disconnect behaviour without exposing a transaction to a different client’s session. Built-in pooling may reduce per-connection cost, but must never mix security contexts or transaction-affined state. A documented binary SQL protocol and a secure HTTP administration API are separate surfaces with separate threat models.

5. Native OLAP processing and high-volume analytics

5.1 Proprietary columnar storage

The original .derc analytical segment concept is retained as the starting point for a versioned Bradian-native column format, with a documented migration from DER terminology. Each segment includes magic/version, schema fingerprint, column directories, typed pages, null bitmaps, dictionaries, compressed buffers, zone maps, offsets and integrity metadata. Immutable row groups enable column pruning and predictable parallel scans; atomic manifest publication makes complete segment generations visible together. Unknown critical format versions must be rejected or opened by a tested compatible reader.

Encoding should adapt to the data: dictionary encoding for repetitive strings, delta encoding for ordered integers, RLE or bit packing for small domains and vetted lossless codecs such as LZ4/Zstd for general payloads. Bradian records compression ratios and decode cost per segment, not just file size. Deleted

and updated rows require version fences, delete vectors or merge-on-write semantics so old analytical copies do not reappear after compaction.

5.2 Vectorized execution and operators

The analytical runtime processes typed column batches with null masks, selection vectors and controlled ownership. Scan, filter, projection, hash aggregation, sort, top-k, distinct, join, window, time-bucket and partial/final aggregation operators execute without materializing every row into heavyweight objects. Supported SIMD paths must be benchmarked and checked against a scalar correctness oracle, including nulls, overflow, floating-point corner cases and collations.

Queries are subject to hard budgets for memory, CPU, scanned bytes, elapsed time, temporary files and output volume. Large joins and sorts spill to managed disk areas rather than terminating the server through memory exhaustion. Query cancellation must release temporary resources, and fair scheduling must keep transaction latency stable while OLAP workloads run in parallel.

5.3 Cost-based optimization and statistics

The optimizer compiles a logical tree and evaluates access paths using row counts, histograms, distinct estimates, sort order, index selectivity, cache state, compression, CPU work, disk throughput, network movement and worker availability. It performs safe predicate pushdown, partition pruning, join ordering, materialized-view substitution and aggregation pushdown. Adaptive feedback can improve future estimates but must never change the answer, bypass authorization or produce nondeterministic schema semantics.

EXPLAIN presents the chosen plan and estimated costs, while EXPLAIN ANALYZE adds actual row counts, operator time, memory, spills, network bytes and skew. Bradian must preserve a conservative fallback plan and maintain property-based equivalence tests for optimizer transformations. Query plans and cached results must be keyed to schema, data revisions, access policy and consistency requirements.

5.4 Materialized views and streaming

The incremental engine reads committed change records, identifies impacted dependencies, retracts old contributions and applies new ones. Updating an event’s date, for example, must decrease the old period aggregate and increase the new one exactly once. Distinct counts, medians, join-dependent measures and other non-invertible states require either maintained sufficient statistics or targeted recomputation. Replaying a message repeatedly must never inflate a count.

Materialized objects publish source watermark, compiler revision, data completeness, schema revision and staleness status. Streaming ingestion must support durable checkpoints, backpressure, malformed-record quarantine, replay and full rebuild. Analytical data products may provide SQL views, aggregates, cohorts, line listings and semantic APIs, but every published product has an owner, lineage, privileges, refresh contract and retirement process.

6. Unified HTAP semantics

6.1 One authoritative commit domain

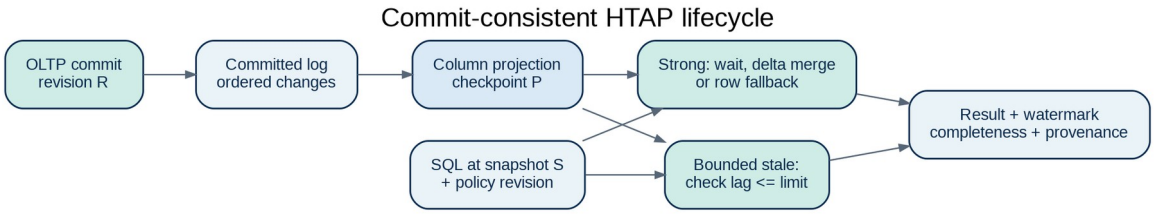


Figure 04_htap: Commit-consistent HTAP lifecycle

Figure: Commit-consistent HTAP lifecycle. Reference design; paths and protocols require implementation and validation.

The defining HTAP requirement is that every analytical representation derives from the same committed transaction history as relational tables. Bradian must maintain a durable change stream from the transaction engine to columnar projections, with monotonically ordered commit coordinates or another provably comparable revision mechanism. No reader should see an uncommitted row because a projection builder executed early. Cross-table projections and indexes require a consistent visibility barrier; a mixture of incompatible commit positions is not a valid strong snapshot.

6.2 Query consistency and projection freshness

The hybrid engine exposes explicit modes: STRONG_SNAPSHOT reads a coherent committed database snapshot; BOUNDED_STALENESS accepts a declared maximum lag; LATEST_AVAILABLE returns the newest safe analytical projection with a visible watermark. For strong queries whose projection is behind, the planner may merge committed row deltas using a correctness-proof protocol, wait within the caller deadline, or use authoritative row storage. It must not silently downgrade consistency.

For one query joining a large columnar fact table and an OLTP dimension table, both sides must resolve to a compatible snapshot and observe the same authorization policy. Projection lag and fallback cost should be observable in the query profile. Materialization errors must not make transactional rows inaccessible or destroy commit history.

6.3 Workload governance and fairness

OLTP, OLAP, ingestion, compaction, replication, backup and schema migrations compete for disk and memory. Bradian must reserve resource headroom for transactional work and enforce workload classes with budgets, priorities and preemption/cancellation policies. A dashboard refresh, large export or remote analytical scan cannot monopolize memory pools, WAL bandwidth or checkpoint I/O. Fairness and service-level objectives are monitored under mixed workload load tests rather than measured only in isolation.

7. Distributed-first horizontal scaling

7.1 Standalone, cluster and federation modes

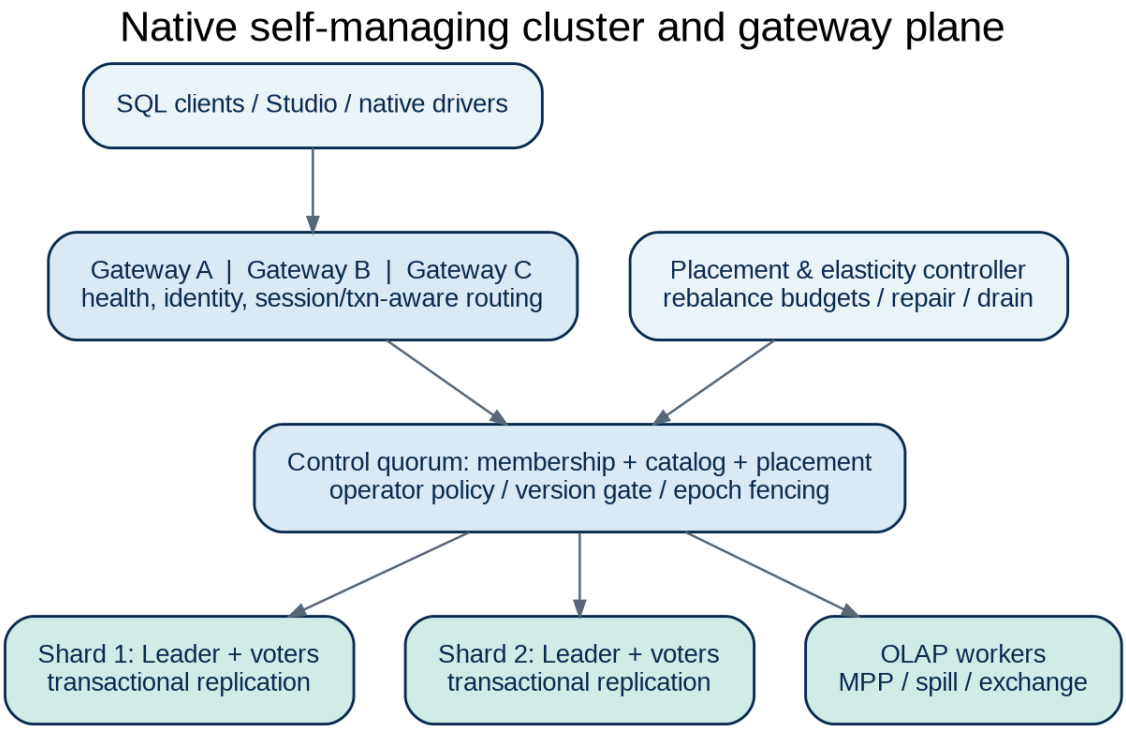


Figure 05_cluster: Native self-managing cluster and gateway plane

Figure: Native self-managing cluster and gateway plane. Reference design; paths and protocols require implementation and validation.

A standalone instance creates a database in one process or service without an external coordinator. A *native cluster* presents one logical strongly governed database across storage shards and replicas connected through reliable infrastructure. *Federated installations* are independent databases that may synchronize selected information or execute privacy-governed cross-site queries. These deployment modes cannot share identical write-availability guarantees: a local installation must not lose all write capability merely because a remote national coordinator becomes unreachable.

The cluster control plane owns membership, role assignment, shard placement, replica health, compatible schema versions and administrative operations. A distributed consensus algorithm can coordinate critical membership and strongly consistent replica state. The design must specify quorum behaviour, split-brain prevention, leader terms, membership transitions and failover fencing before calling itself self-healing.

7.2 Built-in connection load balancers and query routers

Multiple healthy nodes must be able to expose database entrypoints. A client connects through a native gateway or topology-aware driver, authenticates once to the selected session, and is routed toward the correct transaction coordinator or query worker. The connection balancer considers healthy connections and saturation; the transaction router preserves shard ownership, leader epoch and session affinity; the analytical scheduler places fragments close to data; and the storage balancer moves partitions and replicas under controlled capacity policies.

Round-robin is insufficient for distributed transactions. Gateway failover must distinguish safely retryable idempotent requests from uncertain commits. Client drivers should expose a request/transaction ID and an explicit ‘commit outcome unknown’ error where the server cannot establish whether a commit completed. Read routing may use replicas only under the requested consistency contract and replication watermark.

7.3 Sharding and transactional replication

Shard keys and placement schemes should support automatic hash/range partitioning, online split, merge, rebalance and replica placement across independent failure domains. Writes to strongly consistent shards are replicated under a documented commit quorum and fault model. A three-voter shard commonly tolerates one voter failure while retaining a majority, but it cannot promise successful writes after loss of quorum. Data transfer during rebalancing uses checksums, snapshot-plus-catchup and fenced ownership transitions so that a shard is never served by two unsafely active leaders.

Cross-shard transactions require a transaction coordinator and a protocol for atomic commitment, crash recovery and deadlock handling. Consensus at each shard does not itself implement distributed ACID. The design must define commit records, participant recovery, coordinator loss and read visibility across shards. Distributed serializability is a separate, demanding feature gate and must not be claimed merely because transactions work on one shard.

7.4 Multi-site federation and offline reconciliation

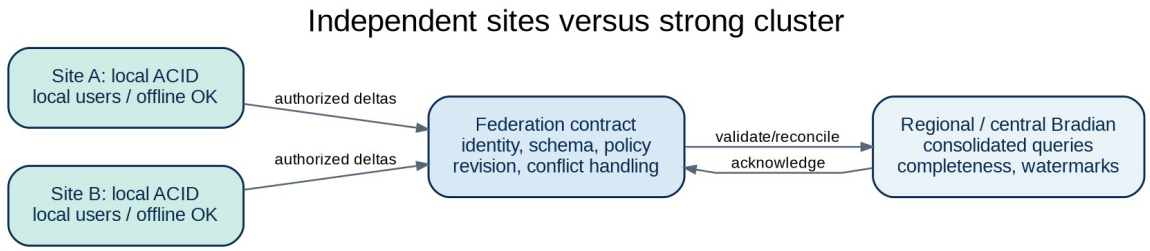


Figure 06_federation: Independent sites versus strong cluster

Figure: Independent sites versus strong cluster. Reference design; paths and protocols require implementation and validation.

Independent Bradian installations can authorize selected tables, streams or aggregate states for exchange. Each outbound record or delta has stable origin, object identity, revision, operation, schema version and replay idempotency. Reconnection performs dependency checks, identity matching and conflict classification; it never guesses one winner for concurrent incompatible changes. Governance policies determine whether raw data, pseudonymized data, aggregates or nothing may leave a site.

Federated analytics marks missing-site coverage and freshness; central distinct counts cannot simply sum site-local distinct people without a valid identity or mergeable state. This capability preserves DER-style offline autonomy while making the general DBMS usable for field offices, branches and regional installations.

7.5 One-command operations and safe automation

The product should provide bradian init, bradian start, bradian cluster join, bradian cluster status, bradian cluster rebalance --plan, bradian cluster drain and related commands through a versioned CLI. Node admission requires mutual authentication, a short-lived join credential, version checks and operator authorization; untrusted discovery beacons must not automatically gain membership. Scaling plans

preview storage movement, expected replica health, risk and rollback limits, while production changes are audited.

Automation must avoid oscillation and destructive ‘self-healing’. Before removing a replica or draining a node, Bradian verifies remaining failure-domain capacity, active snapshots and quorum. Rolling upgrades negotiate wire/storage compatibility and protect against unsafe downgrade. Scaling performance must be measured separately for throughput, query latency, write amplification and network overhead.

8. Portable service architecture and distribution

8.1 Operating-system support

Linux x86-64 is the recommended first production baseline, followed by Windows Server and Windows desktop, then macOS development/server modes and additional Unix-like systems after test evidence. The native runtime provides Windows Service, systemd and launchd integration where appropriate, while preserving the same SQL, storage format and cluster protocol contracts. Package signing, clean installation, uninstall, upgrade and data directory permissions are release-critical. Do not imply that all Unix variants are supported simply because the code is written in a portable language.

8.2 Configuration and secure defaults

A new local instance should start with a loopback-only listener, generated instance identity, no publicly accessible admin account, explicit initial administrator enrollment and protected data-directory permissions. TLS is mandatory for remote database and administrative endpoints, with mutual TLS mandatory for cluster node-to-node communication. Public exposure requires explicit bind configuration and a deployment security check. Secrets must never be written to sample configuration, command history, logs or crash diagnostics.

8.3 Packaging, upgrades and storage evolution

Binary builds must declare OS, architecture, supported filesystem, format version, protocol version and tested upgrade paths. The storage engine needs reversible metadata migrations when possible, backup requirements for irreversible format changes, safe online upgrade gates and explicit refusal of unsupported downgrade. Dependency manifests and signed release artifacts are mandatory; an installation must be able to verify update provenance before replacing an executable.

9. Bradian Studio: browser-based database administration

9.1 Product boundaries and navigation

Bradian Studio is a separate, secure web application served by or connected to Bradian, not a hidden developer console. It includes Server Overview, Database Explorer, Table Designer, Data Browser, SQL Workbench, Analytics, Data Products, Query Profiler, Cluster, Replication, Security, Backups, Audit, Monitoring, Jobs and Settings. Each route has a distinct job: lists, object details and create/edit forms should be separate pages, while non-editing inspection can use an optional detail drawer. The global navigation must remain compact and predictable.

The visual style follows the approved Bradian identity: white and neutral surfaces, navy headings, restrained teal/blue functional accents, no gradients, readable typography, balanced tables and minimal decorative boxes. The approved horizontal logo should be used in page chrome at a modest size, with the single-colour white variant for dark backgrounds. UI layouts must work on Windows, macOS and Linux browsers and support keyboard interaction and accessible labels.

9.2 SQL Workbench and data explorer

The Workbench offers multiple query tabs, syntax highlighting, schema-aware completion, parameterized execution, cancellation, explain/profile views, plan comparison, result paging, query history and controlled exports. It must separate read-only inspection from data-changing execution with permissions and confirmation for destructive operations. Results are streamed or paged with budgets, never fully downloaded into the browser by default. Sensitive values are masked according to policy even in explain/profile and saved query history.

The Explorer navigates databases, schemas, tables, columns, constraints, foreign keys, indexes and views. Table detail shows schema, storage statistics, partitions, dependencies and permissions. Data editing occurs on a dedicated page or explicit transaction editor and honors row-level authorization, optimistic revision checks and constraint errors; an administrator UI must not bypass the database transaction engine.

9.3 Operational consoles and diagnostics

Cluster screens display actual nodes, roles, shard leaders, replica lag, quorum risk, placement, load balancing, in-flight relocations and upgrade progress. Security screens manage users, roles, grants, sessions, certificates, key rotation and audit evidence with protected workflows. Backup screens show snapshot verification and restore rehearsal results, not just filenames. Performance screens link slow queries to plan operators, storage pages, lock waits and workload admission. Every operation returns auditable action ID, outcome and recovery guidance.

9.4 Studio security baseline

Studio requires secure session cookies, CSRF defense, strong authentication/MFA for privileged users, content security policy, output encoding and strict API authorization. Browser tokens may not grant direct filesystem access, shell execution or broad cluster control. The web frontend must use parameterized query endpoints with explicit statement permissions and enforce authorization again at execution. An administrative action cannot become safe solely because its button was hidden in the UI.

10. Security architecture and non-negotiable trust model

10.1 Security objectives and threat boundaries

Security is a core database correctness property, not a feature added after SQL works. Bradian protects confidentiality of records, integrity of committed transactions and availability of authorized workloads against network attackers, compromised clients, malicious insiders, misconfigured nodes, untrusted imports, storage corruption and software supply-chain compromise. The threat model covers the public SQL listener, Studio and administrative APIs, cluster join, replication transport, local disk, backup destinations, third-party connectors, plugins, control plane and operational staff. A privileged database identity is not automatically a privileged operating-system identity.

A product security programme must maintain data-flow diagrams, trust boundaries, abuse cases, asset classification, security requirements, remediation SLAs and test evidence. Explicit trust zones include client-to-gateway, gateway-to-coordinator, node-to-node replication, control-plane membership, backup storage, federation endpoints and build-to-release artifacts. Each boundary must authenticate the counterparty, authorize its requested action, validate untrusted input and produce meaningful audit events. Default deny is the baseline.

10.2 Identity, authentication and credential lifecycle

Bradian must integrate local identities and enterprise identity providers through supported, tested methods. Administrative login should require phishing-resistant MFA where feasible, with strong

enrollment/recovery controls and time-bounded emergency access. Database drivers use secure password authentication, client certificates or approved delegated tokens; weak plaintext methods are not enabled by default. Passwords are stored only through contemporary password-hashing schemes with unique salts and tuneable work factors, never reversible encryption. Session and refresh tokens must have audience, lifetime, issuer, revocation and rotation semantics.

Provisioning and deprovisioning workflows must be auditable. A disabled identity must stop creating sessions, and sensitive long-running jobs or exports must recheck authorization before delivering results. Service accounts have explicit owners, narrow scopes and expirations; node identities are separate from human administrators. Certificate renewal and clock-skew handling must be rehearsed before deployment. Credential secrets cannot appear in server logs, query plans, process arguments or support bundles.

10.3 Authorization: RBAC, ABAC, row and column security

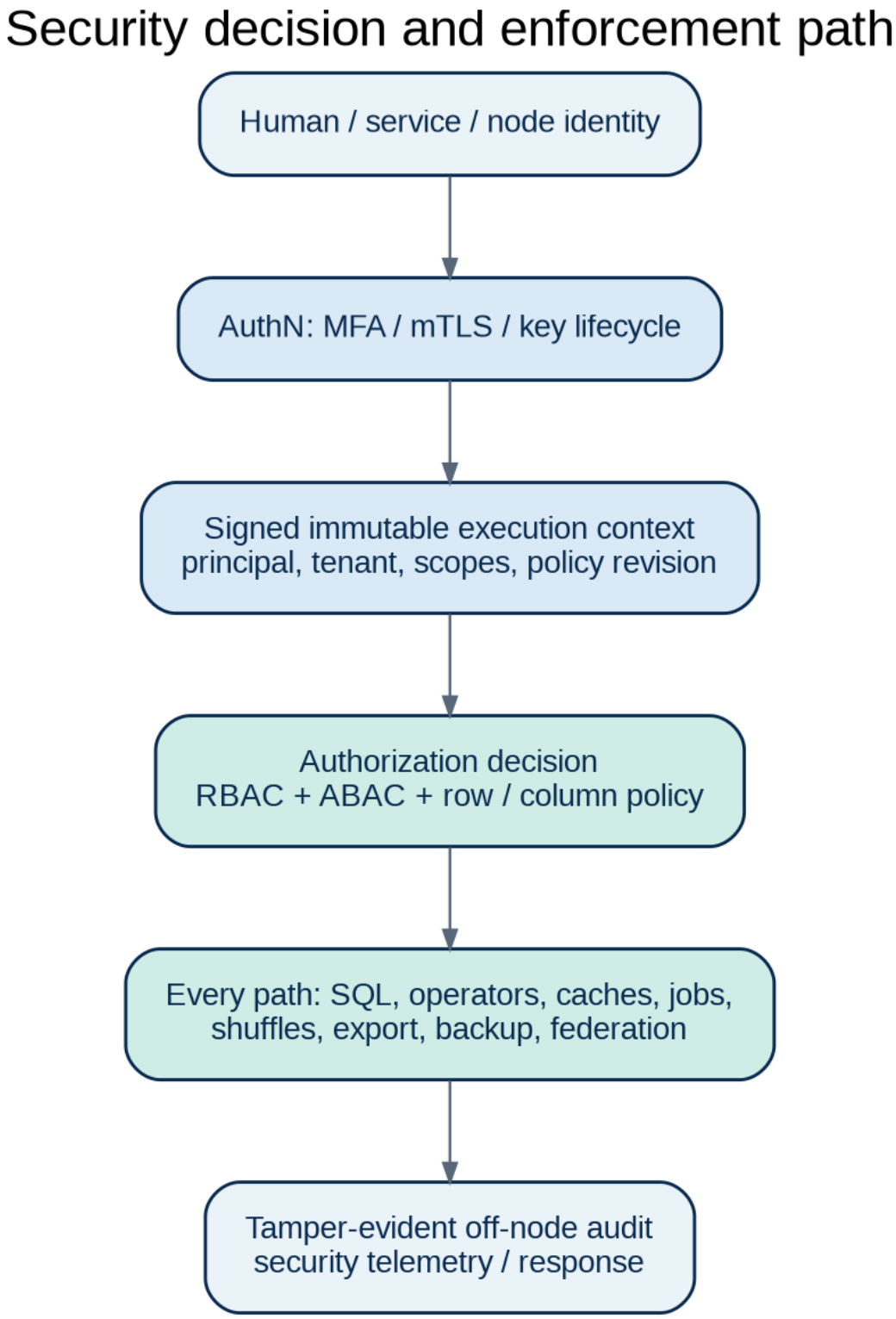


Figure 07_security: Security decision and enforcement path

Figure: Security decision and enforcement path. Reference design; paths and protocols require implementation and validation.

The authorization model should provide database, schema, table, view, column, function, sequence, job, backup, cluster-operation and audit-reader privileges. Role-based access control supplies manageable grants; attribute-based conditions allow site, organization, classification, ownership, device trust or environmental context. Row-level policies and column masking are bound into query planning and enforced at runtime, including joins, nested queries, materialized views, data exports, background jobs and cache reads.

Policies must be impossible to bypass using a lower-level analytical API, a foreign table, a restore preview or EXPLAIN. The system should provide explicit definer/invoker semantics for views and routines, with strict controls on privilege escalation. Every cached result is isolated by effective security context, policy revision, schema revision and snapshot. Revocation must invalidate relevant sessions, caches and pending exports according to a documented maximum enforcement delay.

10.4 Tenant isolation and multi-database separation

A deployment may contain multiple tenants, departments or customers. Bradian must define tenant identity independently of mere database naming; policy decisions and cryptographic domains must never rely on a user-controlled string alone. Default deployments isolate tenant catalogs, data directories or object namespaces, encryption keys, cache entries, temporary spills, logs and backups according to the selected isolation profile. Cluster placement and analytical exchanges must respect tenant boundaries before optimizing cost.

Resource isolation matters as much as access control: one tenant’s expensive query must not starve another tenant’s transactional availability. Tenant-aware quotas cover connections, CPU, memory, temporary disk, storage, network transfers, background jobs and export sizes. Security tests must attempt cross-tenant row access, cache poisoning, metadata inference, query-plan leakage, backup mix-ups and privileged operation confusion.

10.5 Encryption in transit and at rest

All remotely accessible database and Studio endpoints must use maintained TLS configurations with certificate validation, appropriate protocol versions and strong cipher suites. Cluster RPC and federation channels require mutual TLS plus explicit node authorization. Avoid silent plaintext fallback, insecure certificate skipping and unauthenticated discovery-to-join shortcuts. Session resumption, certificate rollover and key compromise response must be implemented without disrupting legitimate access unexpectedly.

At-rest protection includes database files, WAL, snapshots, manifests, temporary spill files and backups. Bradian should use envelope encryption with keys derived or unwrapped through an approved KMS/HSM or protected local keystore. Distinct key-encryption keys and data-encryption keys support controlled rotation. Encryption metadata and integrity tags must remain versioned and authenticated, and keys must never be embedded in binary artifacts or source control. Crypto primitives must come from reviewed libraries; proprietary encryption algorithms are prohibited.

10.6 Key management and secret custody

A formal key hierarchy defines root/master keys, per-instance or per-tenant key-encryption keys and rotated data keys. Secret storage uses operating-system facilities or supported external secret managers, with least-privileged retrieval and audited rotation. Recovery keys must be documented and access-controlled; losing all copies of a required key must not be presented as recoverable by routine restore. The product must support safe key rollover and re-encryption progress reporting without exposing plaintext to logs or untrusted plugins.

Key compromise playbooks require certificate revocation, session invalidation, emergency rotation, affected-backup assessment and incident reporting. A KMS outage must have a defined policy (for example, bounded cached key use for existing encrypted pages) rather than an undocumented fail-open mode. Bradian must accurately describe what encryption does and does not protect when the host kernel or a privileged process is compromised.

10.7 Secure transport, query and input surfaces

SQL parsing, binary frame decoding, HTTP JSON, CSV ingestion, native segment readers and extension loaders are untrusted-input boundaries. Parsers must enforce length bounds, recursion limits, integer overflow checks, allocation limits and strict schema validation before accepting data. Parameterized statements prevent SQL injection through application drivers; Studio must never concatenate untrusted form fields into SQL. Query text containing secrets must be redacted from telemetry, saved statements and diagnostics where possible.

SQL execution cannot invoke arbitrary operating-system commands, load uncontrolled dynamic libraries, read arbitrary host paths or open outbound sockets unless a narrowly scoped, reviewed extension explicitly receives such capability. Imported files are staged in safe directories and validated. Decompression bombs, huge schema metadata, corrupt indexes and adversarial query plans must fail in a bounded manner, preserving service availability.

10.8 Audit, evidence integrity and privacy

The audit subsystem records authentication, grants and revocations, schema mutations, SQL access to protected objects, data exports, backup/restore, key rotation, cluster membership changes and security configuration updates. Each event identifies actor, effective identity, source, request/correlation ID, target, policy decision, timestamp, node and outcome; sensitive row values are not copied indiscriminately into audit logs. Audit retention and access are governed separately from ordinary query logs.

For high-assurance environments, audit records should be integrity protected using append-only or hash-chained batches with external checkpointing or equivalent tamper evidence. This does not make a fully compromised administrator incapable of deleting every local log, so deployments should support prompt off-node export to protected audit storage. Privacy controls include retention, deletion requests, minimization, masking, lineage for derived views, backup expiry and data-residency checks. Legal compliance must be assessed per deployment and jurisdiction rather than assumed from a technical feature list.

10.9 Cluster and federation security

A new Bradian node may join only through an authenticated enrollment workflow, with certificate identity, signed or protected join token, explicit allowed role, cluster ID, protocol version and admission policy. Peer discovery does not itself grant authority. Compromised, stale or removed replicas must be fenced from accepting writes. Consensus metadata, shard transfer and replica traffic use mutual authentication and replay protection; quorum loss must never trigger an unsafe automated split-brain promotion.

Federation has an additional trust boundary: each participating installation decides which information can be exported and under what policy. Site-to-site links need a federated identity, scopes, dataset contracts, schema compatibility, encrypted transfer, idempotent replay and audit. A query optimizer cannot send protected patient-level or commercially restricted data to another jurisdiction merely because remote computation is cheaper. Remote health status and result completeness must be visible to operators and users.

10.10 Platform and host hardening

Bradian should run as a dedicated unprivileged service account, with minimal filesystem rights, protected data/config directories, restricted network listeners and optional sandboxing. Linux hardening should address systemd sandbox directives, privilege dropping and capabilities; Windows packaging should use suitable service identities and filesystem ACLs. Administrative access must be separated from database workload access. No installation should ship universal default credentials, open remote superuser access or an unauthenticated Studio endpoint.

The configuration validator should identify exposed listeners, disabled encryption, overly broad grants, weak passwords, missing backups, unsupported operating systems, unsafe plugins, expired certificates and excessive retention. Operators may inspect a machine-readable hardening report before production go-live. Bradian must document the shared-responsibility boundary: the product cannot compensate for an unpatched compromised host or insecure physical controls.

10.11 Software supply chain and secure development

The repository requires signed commits or release provenance where feasible, protected branches, independent code review for security-critical paths, dependency pinning, SBOM generation, license review and automated vulnerability scanning. CI secrets are scoped and short-lived; release signing keys are segregated from ordinary build agents. Reproducible or verifiable builds are preferred. Cryptographic code, WAL recovery, cross-shard commit, parser, authorization and key custody receive threat modeling and focused peer review.

Continuous testing includes SAST, dependency/SBOM scanning, dynamic security tests of Studio, fuzzing SQL and binary segment parsers, adversarial transaction schedules, sanitizer builds, TLS downgrade tests and isolated penetration tests. Release policies must define critical vulnerability response time, patch channels, supported versions and disclosure procedures. A high benchmark score never waives a security or data-integrity failure.

10.12 Security incident response and assurance gates

Operators need incident signals for credential abuse, excessive failed logins, privilege escalation, anomalous exports, certificate failure, audit disruption, replication tampering, unexpected code loading and suspicious network activity. The product should generate structured alerts while limiting false positives and preserving service availability. Incident workflows include isolate or drain a node, revoke credentials, retain forensic evidence, rotate keys, identify data exposure and restore from validated clean state.

The security acceptance baseline should map controls to NIST SP 800-53 Rev. 5, OWASP ASVS 5.0 for Studio/API components, and relevant CIS platform benchmarks; these are design and testing references, not certifications. Independent security assessment, cryptographic verification, documented exceptions and remediation of critical findings must precede production claims of high assurance. **No platform can responsibly promise absolute security**; Bradian must instead provide a defensible security programme with measurable controls and continual review.

11. Recovery, backup, replication and data lifecycle

11.1 Backup architecture

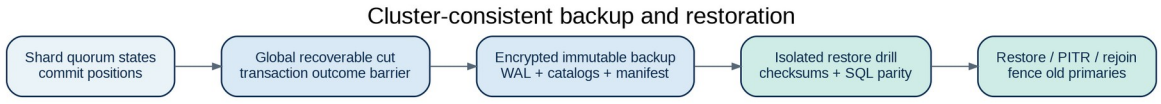


Figure 09_recovery: Cluster-consistent backup and restoration

Figure: Cluster-consistent backup and restoration. Reference design; paths and protocols require implementation and validation.

A transaction-consistent backup includes an authoritative data snapshot, catalog state, WAL position, identity/key references and restore manifest. Bradian should support full and incremental backups where the format permits, encryption before remote transfer, authenticated integrity metadata, retention policies and offline restore verification. Backup success means the data can be restored and queried correctly, not merely that bytes were copied. Operators should regularly run automated restore rehearsals into isolated environments.

11.2 Point-in-time recovery and disaster recovery

Point-in-time recovery replays the transaction log from a verified snapshot to a selected committed boundary while excluding uncommitted or incomplete operations. Recovery logic must resolve in-doubt distributed transactions according to its durable coordinator records; it must not arbitrarily commit or abort a transaction whose true outcome can be established. RPO and RTO targets are deployment-specific service objectives with documented measurement conditions. Regional disaster recovery must account for replica consistency, network partition and key custody.

11.3 Retention, deletion and compaction

A deletion must propagate to transaction versions, indexes, columnar projections, materializations, caches, federated replicas and eventual backup expiry, subject to legal holds and retention policies. Background vacuum, compaction and tombstone collection cannot discard data still visible to active snapshots or needed for safe recovery. Operators need a data lineage view showing where derived values exist and whether a deletion is pending, applied or blocked by a stated policy.

12. Data integration, interoperability and extension model

12.1 Client protocols and drivers

The target platform requires a versioned SQL wire protocol and first-class JDBC, ODBC and one or more language drivers. Protocol design must specify authentication negotiation, prepared statements, type encoding, streaming cursors, transaction boundaries, deadlines, cancellation and structured error codes. Proxy support and topology-aware connection routing must work without requiring Studio. Compatibility with other database protocols may be offered only after exact behaviour and limitations have been documented.

12.2 Bulk ingestion and export

Bradian must support controlled CSV, JSON and structured interchange and SHOULD add Arrow IPC and Parquet through reviewed libraries or self-contained readers. Ingestion jobs validate schema mappings, data type coercion, batch transaction scope, duplicate policies, quarantine and restart checkpoint. Large exports are asynchronous, paged or streamed, permission checked at release, rate limited and audited. External tables and connectors operate under explicit network and credential scopes.

12.3 DER, DHIS2 and specialized extensions

DER and DHIS2 support is a separately versioned adapter family rather than an implicit built-in schema. The adapter translates aggregate values, category combinations, Events, Trackers, Enrollments, Relationships, indicators, organisation hierarchies, periods and revisions into generic Bradian storage and semantic contracts. It preserves DHIS2 authority and user permissions and reports unsupported expression semantics explicitly. Other vertical integrations may follow the same plugin architecture without modifying the database kernel.

12.4 Extensibility boundaries

Extensions may add scalar functions, data connectors, storage codecs or analytical operators only through a versioned and capability-restricted API. Prefer sandboxed extension processes or memory-safe plugins for untrusted code; in-process native plugins require an explicit trust designation and version negotiation. Extensions cannot bypass authorization, impersonate the core, access arbitrary files or participate in consensus without a reviewed protocol. Failed plugins must not corrupt catalog or committed data.

13. Operations, telemetry and performance management

13.1 Observability and explainability

Bradian exposes metrics for transaction commits, aborts, lock waits, deadlocks, replication lag, quorum health, checkpoint age, WAL bytes, cache hit rate, row/column scan bytes, materialization lag, ingest queues, memory pressure and query p50/p95/p99. Traces correlate client statements with gateway routing, planner decisions and worker fragments without revealing sensitive parameter values. The query history preserves fingerprints and redacted plans under retention controls.

13.2 Workload and admission control

Administrators define workload classes for interactive transactions, BI queries, batch analytics, ingestion, compaction, backup, restore and migrations. Each class has reservations and limits for CPU, memory, concurrency, temporary storage, I/O and network. Admission control must reject or queue work with understandable diagnostics rather than permit memory collapse. Auto-tuning suggestions use measured statistics and require safe rollback; learning algorithms cannot change SQL truth or override security policy.

13.3 Service objectives and benchmark discipline

The original DER benchmark goals provide useful *aspirational baselines* for materialized query latency and ingestion freshness, but Bradian must establish new mixed-OLTP/OLAP benchmarks including transaction throughput, commit latency, distributed write contention and cross-platform crash durability. All metrics require pinned hardware, dataset, concurrency, cache state, consistency and error bars. A performance comparison is invalid if Bradian returns stale results while the reference system returns strong snapshots, or if security checks are disabled on one side.

14. Non-functional requirements and preliminary service targets

14.1 Reliability and correctness

The release criterion for acknowledged data loss under the documented durability mode is zero in the tested failure model. The engine must preserve constraints, serializability where advertised, storage checksums, recovery and analytical parity under concurrent load. Standalone production release requires crash tests at commit and checkpoint boundaries; distributed release adds leader death, stale gateway, network partitions and reconfiguration fault matrices.

14.2 Initial measurable engineering gates

Relational correctness — 100% passing SQL/transaction reference corpus
Verification context: Documented SQL subset and isolation mode
Durable acknowledgement — No acknowledged transaction lost in supported crash model
Verification context: Kill/restart/power-loss simulation and WAL checks
Mixed workload isolation — OLTP p95 regression below 10% at declared OLAP co-load
Verification context: Same server, same data and admission policy

Interactive analytical result — p95 under 2 seconds on published 100M-fact fixture
Verification context: Fixed hardware/query/concurrency/cache conditions
Incremental projection — p95 visibility under 5 seconds in local nominal profile
Verification context: Commit-to-visible watermark
Scale-out throughput — Demonstrable growth with added workers before bottleneck
Verification context: Publish scaling efficiency and partition layout
Security authorization — No unauthorized row, schema, export or backup disclosure in test matrix
Verification context: RBAC/ABAC/tenant and revocation tests
Restore — Verified data and catalog checksum parity
Verification context: Full snapshot + WAL restore rehearsal
Availability — Explicit quorum/fault-domain envelope; no blanket 100% claim
Verification context: Replica loss, network split, gateway failover

These targets are engineering hypotheses rather than certified service levels. A measured deficiency is a prioritised optimization task, never permission to weaken isolation, security, constraints or correctness.

14.3 Accessibility, operability and documentation

Studio should target WCAG 2.2 AA where practicable and test keyboard, screen-reader labels, contrast, focus management, large tables and responsive layouts. CLI messages and API errors should be actionable, stable and non-sensitive. Every public setting requires documented default, risk, allowed range, reload/restart behaviour and upgrade compatibility. Administrative guides must explain not only what action to perform but what failure modes it protects against.

15. Implementation programme and release gates

15.1 Engineering sequence

Gate 0 — Repository and architecture audit. Inspect the existing dap repository and supplied DER analytics code/documents; map source dependencies and identify what is functional versus planned. Produce a requirements traceability matrix, threat model, architectural decision records (ADRs), executable test harness and baseline CI. Do not replace valid code merely to align naming.

Gate 1 — Portable relational kernel. Implement native server startup, data directories, generic type/catalog, portable row pages, basic B+tree and a minimal SQL CREATE/INSERT/SELECT slice. Exit only with persisted data, restart durability, format tests and a basic driver or CLI.

Gate 2 — Correct transactions. Add WAL, atomic commit, rollback, MVCC, locks/conflicts, constraints, crash recovery and SQL DML. Exit with concurrent anomaly tests, fault injection, recoverable interrupted transactions and audited data/permission semantics.

Gate 3 — Full core relational SQL. Implement joins, subqueries, views, schema migrations, composite constraints, prepared statements, transactional DDL policies and explain plans. Test against a published reference SQL corpus and supported isolation levels.

Gate 4 — Native columnar/OLAP engine. Preserve and port the original .derc column format, vector batches, compression, indexes, parallel operators, CBO, spill and incremental views. Validate against the scalar oracle and real large-data benchmarks.

Gate 5 — Unified HTAP. Establish commit-coordinate propagation, coherent cross-store snapshots, bounded staleness, strong fallback, commit-aware materialization and mixed workload reservations. Fail if an analytical plan can return mismatched revisions.

Gate 6 — Bradian Studio and external clients. Ship browser administration, secure SQL Workbench, database explorer, schema designer, query profiles, audit, backup and node diagnostics. Deliver supported JDBC/ODBC/native client contracts, not UI-only functionality.

Gate 7 — Native horizontal scaling. Implement secure membership, multiple gateways, shard routing, consensus replication, transactional failover, online rebalancing and at least a defined subset of cross-shard atomic transactions. Exit with split-brain and cluster fault evidence.

Gate 8 — Federated installations and portable packaging. Implement independently operable site databases, secure replication/federation, Windows and Unix support, installers, backup migration and compatibility/upgrade testing.

Gate 9 — Security and production qualification. Complete threat remediation, fuzzing, penetration tests, SBOM/provenance, recovery drills, rolling upgrades, data migration and benchmark publication. No release label may claim features that lack the corresponding gate evidence.

15.2 Subsystem implementation deliverables

Every gate produces working source, unit and integration tests, API/schema updates, traceability status, documentation of known limitations, reproducible benchmark artifacts, a rollback/upgrade plan and a changelog. Security-critical modules additionally require independent reviewer sign-off. The implementation agent must report blocked items truthfully and avoid describing scaffolds, unused interfaces or sample screens as production-ready.

16. Formal functional requirement register

BRD-REL-001 — Native databases, schemas, relational tables and stable catalog IDs
Verification: SQL schema integration
BRD-REL-002 — PK, FK, UNIQUE, CHECK, defaults and referential actions
Verification: Constraint and race corpus
BRD-REL-003 — DDL, DML, prepared statements and documented SQL subset
Verification: Differential SQL tests
BRD-REL-004 — Typed JSON, decimals, timezones and correct NULL semantics
Verification: Type golden corpus
BRD-TXN-001 — ACID, commit acknowledgement, rollback and savepoints
Verification: Crash and transaction tests
BRD-TXN-002 — MVCC snapshots and implemented isolation levels
Verification: Concurrent anomaly tests
BRD-TXN-003 — Deadlock handling, lock timeouts and revision GC
Verification: Stress and long-snapshot tests
BRD-STO-001 — Native row pages, primary/secondary indexes and checksums
Verification: Storage format tests
BRD-STO-002 — Versioned native columnar segments and atomic manifests
Verification: Segment/read/corruption tests
BRD-STO-003 — WAL, checkpoints, restore, PITR and snapshot consistency
Verification: Recovery drills
BRD-OLAP-001 — Vectorized and parallel SQL operators with bounded spills
Verification: Oracle and low-memory tests
BRD-OLAP-002 — CBO, statistics, EXPLAIN and plan equivalence
Verification: Rewrite property tests
BRD-OLAP-003 — Incremental views with edits, deletes and replay safety
Verification: Contribution/retraction tests
BRD-HTAP-001 — Coherent committed snapshots across row/column stores

Verification: Cross-store revision tests
BRD-HTAP-002 — Strong, bounded-staleness and latest-safe query contracts
Verification: Lag/fallback tests
BRD-HTAP-003 — OLTP resource protection during concurrent analytics
Verification: Co-load benchmark
BRD-CL-001 — Secure bootstrap, join, membership and leader fencing
Verification: Admission and split-brain tests
BRD-CL-002 — Native multi-gateway routing and safe retry semantics
Verification: Gateway fault tests
BRD-CL-003 — Sharding, replicated write quorum and online rebalancing
Verification: Placement/failover tests
BRD-CL-004 — Distributed transaction and coordinator recovery
Verification: Cross-shard crash matrix
BRD-FED-001 — Independent site operation and authorized federation
Verification: WAN outage/convergence tests
BRD-FED-002 — Data residency and version-aware conflict resolution
Verification: Privacy/replay tests
BRD-SEC-001 — Authentication, MFA-capable privileged login and secure sessions
Verification: AuthN threat tests
BRD-SEC-002 — RBAC/ABAC, row/column/tenant policies end-to-end
Verification: Unauthorized-access matrix
BRD-SEC-003 — TLS/mTLS and managed at-rest encryption
Verification: Protocol/crypto tests
BRD-SEC-004 — Versioned secrets, keys, rotation and revocation
Verification: Rotation/compromise drills
BRD-SEC-005 — Tamper-evident audit and protected export
Verification: Integrity and audit tests
BRD-SEC-006 — Fuzzed parsers, plugin isolation and secure host defaults
Verification: Fuzz/SAST/DAST suite
BRD-SEC-007 — Verified incident/backup/recovery security workflows
Verification: Tabletop and restore exercise
BRD-STU-001 — Browser database explorer, data editor, SQL workbench
Verification: UI E2E and auth tests
BRD-STU-002 — Cluster, security, backup and performance consoles
Verification: Operational E2E tests
BRD-API-001 — Versioned SQL drivers and secure administrative APIs
Verification: Contract and load tests
BRD-OPS-001 — Observability, limits, cancellation and upgrade support
Verification: Reliability/load tests
BRD-PORT-001 — Native Windows/Linux/macOS profiles and storage portability
Verification: Platform CI/installation tests
BRD-EXT-001 — Controlled plugins and optional DER/DHIS2 adapter
Verification: Capability and compatibility tests

17. Comprehensive acceptance and adversarial test scenarios

17.1 OLTP and SQL

A payment transaction that inserts a parent, several children and a ledger adjustment must either commit all changes or leave none, even if the process crashes between WAL flush and page write. Concurrent clients inserting the same unique key cannot both succeed. A foreign-key delete concurrent with a child

insert cannot leave an orphan. Long-running snapshots must remain consistent through updates and online index creation. Documented isolation levels must reject or serialize known anomalies instead of merely passing single-thread tests.

17.2 OLAP and HTAP

A high-volume analytical query must produce exactly the same supported result as the scalar reference engine under nulls, late events, deletes and repeated updates. A materialized view replayed from duplicate journal changes cannot double count. A strong query joining row and column data must use compatible commit positions even if projection building is delayed; a bounded-stale query must reveal its age. During full analytical load, priority transactional requests remain within the declared protection threshold.

17.3 Distributed reliability

A gateway may fail during a transaction, but its replacement must not silently submit a non-idempotent write twice. A shard leader lost before acknowledgement must trigger a safe outcome; a stale former leader must not accept new committed writes. Rebalancing must not lose rows, violate uniqueness or expose mixed snapshots. Loss of quorum must stop unsafe strong writes rather than split the cluster. A federation disconnected for days must preserve each site’s permitted local transactions and reconcile authorized changes with explicit conflicts.

17.4 Security and privacy

A user forbidden from a tenant’s table cannot retrieve its values through SQL, a cached result, a materialized view, backup preview, EXPLAIN, federated join or export. Revoking access during a long-running export must prevent an unauthorized final release. A fabricated node-join request, expired certificate, swapped backup, corrupted segment and malicious plugin must be rejected without privilege escalation. Failed login and unusual export activity must generate protected evidence without revealing credentials or sensitive payloads.

17.5 Portability and operations

A database created on one supported operating system must open through a tested portable format reader on another, subject to declared path/encoding constraints. Recovery must survive interrupted upgrades and disk-full checkpoint failure. Bradian Studio must not allow an unauthorized button or crafted API request to execute an administrative operation. Signed installers must be verifiable, and a fresh deployment must not expose public administration by default.

18. Reference documents, standards and design influences

The following resources inform the architecture; they are not claims of certification, nor mandatory dependencies on external database systems.

Source baseline: *DER Analytics Platform — Comprehensive Product, Storage, Compute and Distributed Analytics Architecture*, v1.0 (11 November 2026), supplied by the product owner. Original native columnar, planning, execution, incremental and distributed analytics requirements.

Security controls: NIST SP 800-53 Rev. 5, *Security and Privacy Controls for Information Systems and Organizations*: https://csrc.nist.gov/pubs/sp/800/53/r5/final

Web/API security verification: OWASP ASVS 5.0.0: https://asvs.owasp.org/

Host/database hardening: CIS Benchmarks: https://www.cisecurity.org/cis-benchmarks

Secure development: NIST Secure Software Development Framework, SP 800-218: https://csrc.nist.gov/pubs/sp/800/218/final

Database concurrency design reference: PostgreSQL MVCC documentation: https://www.postgresql.org/docs/current/mvcc.html

Columnar analytical reference: ClickHouse architecture and MergeTree documentation: https://clickhouse.com/docs/

Distributed SQL architecture reference: CockroachDB architecture documentation: https://www.cockroachlabs.com/docs/stable/architecture/overview

Data interchange: Apache Arrow specification: https://arrow.apache.org/docs/format/Columnar.html

Web accessibility: W3C WCAG 2.2: https://www.w3.org/TR/WCAG22/

19. Claude engineering mandate and definition of done

Claude or any implementation team must first read this master document and the supplied baseline, audit the existing source, inventory completed work, establish requirements-to-tests traceability, and implement one verified vertical slice at a time. Design decisions concerning file format, isolation, WAL, replication, distributed commit, encryption, plugins and SQL compatibility must be recorded as ADRs before irreversible code or persistent format decisions. Every release must expose its implemented capabilities and unsupported features accurately.

Definition of done: production-relevant features are functional in code, integrated through the native service and supported drivers, protected by authorization, covered by correctness and crash tests, documented for operators, benchmarked where performance is claimed, and operable through supported command-line and browser workflows. Incomplete stub services, a nonfunctional cluster diagram, an unaudited crypto feature or an attractive Studio screen are not deliverables by themselves.

Document status statement: This master specification establishes the intended scope and engineering acceptance conditions for Bradian Database Systems. It does not certify the security, performance, interoperability, durability or existence of any implementation. Those claims require measured release evidence.

PART II — VERSION 1.1 ENGINEERING DESIGN ADDENDUM

20. Architecture invariants, specification hierarchy and decision ownership

20.1 Invariants enforced in code

INV-001 Commit correctness. No transaction is acknowledged as durable until the chosen local and, where requested, quorum durability obligations are satisfied. This guarantee applies to catalog changes, table writes, foreign-key side effects and internal transaction outcome records. Optimizations may reorder work only if the resulting observable histories remain valid for the advertised isolation level.

INV-002 One authoritative state. Authoritative records are committed transactional row versions and their durable history. Columnar projections, caches, indexes, materializations and external replicas are recoverable derivatives or protocol-defined replicas; no independent analytics writer may silently overwrite authoritative rows. A query combining data structures MUST bind all inputs to a consistent snapshot or explicitly declare bounded staleness.

INV-003 No authorization gap. Every access path—including distributed operators, export jobs, backups, explain plans and restored previews—MUST evaluate an authenticated and versioned authorization

context, or demonstrably inherit a narrower, validated decision. Metadata and row contents may not leak through alternate introspection paths.

INV-004 No unsafe automatic availability. A strongly consistent shard that loses quorum MUST reject writes requiring the quorum. The automatic controller cannot manufacture quorum by simultaneously promoting disconnected former leaders. Autonomous federation remains a distinct model with explicit reconciliation obligations.

INV-005 Recoverable lifecycle. Every persistent format, catalog migration, schema job, shard relocation and encryption-key transition MUST have an enumerated state machine, durable progress markers, restart behavior, observability and either a safe rollback or a documented forward-recovery path.

20.2 Specification precedence and ADR gates

The architecture is divided into an invariant layer (data safety, authorization and correctness), protocol layer (wire, storage and transaction contracts), product layer (SQL, administration and analytics), and deployment layer (platform packaging and automation). No lower-level change may violate an invariant to achieve a performance target. Architectural decisions for consensus, global timestamps, locking/MVCC, distributed atomic commit, WAL recovery, collations and key custody MUST receive signed ADRs stating considered alternatives, trade-offs, recovery proof obligations and migration impacts. Draft or unimplemented features remain clearly identified in capability manifests; operators MUST NOT be presented with a fabricated complete feature list.

21. Distributed global transaction, timestamp and recovery specification

21.1 Atomicity across shards

A logical transaction obtains an opaque globally unique identifier, participating shard set and versioned coordinator record. Each shard enforces local serializability or documented supported isolation before voting that it can commit. The coordinator persists the global outcome using a fault-tolerant record service; each participant makes the decision locally durable before exposing effects that depend on the completed commit. A two-phase commit (2PC) family of protocols may be selected, but 2PC without replicated coordinator outcome records introduces unacceptable coordinator-failure ambiguity. Consensus replication of each shard is distinct from cross-shard commit coordination and is not a substitute for it.

How to implement: (1) define a TxnId, coordinator epoch, participant set, coordinator lease and canonical serialized outcome; (2) expose internal Prepare, Decide, Resolve and GetOutcome RPCs with authenticated idempotency keys; (3) persist prepare intent and lock/conflict metadata; (4) durably record the single global decision; (5) apply commit/abort and publish a visibility barrier; (6) permit re-drive by a replacement coordinator using a higher fenced epoch. An UNKNOWN_COMMIT_OUTCOME client response MUST be resolvable by transaction ID without replaying non-idempotent application writes.

21.2 Global snapshot and timestamp ordering

Bradian must choose and document one of the following viable approaches before claiming distributed serializable SQL: a globally allocated logical timestamp oracle with explicit failure and monotonicity semantics; a hybrid logical clock combined with uncertainty and validation; or a proven deterministic ordering protocol. Wall clocks alone cannot define safe serializability. A global read snapshot needs a commitment or safe-time rule describing what each participating shard has applied, along with read uncertainty and a policy for unavailable shards.

How to implement: define a CommitPosition abstraction carrying shard epoch and commit sequence, a GlobalSnapshot vector or scalar interpretation, and deterministic rules for read timestamp validation. Establish invariant tests on skewed clocks, delayed follower application, leader changes and concurrent transactions across shards. On a follower, snapshot reads may proceed only when its published applied

watermark satisfies the requested snapshot. Schema catalog revisions and security-policy revisions are part of the same snapshot context.

21.3 Deadlocks, retries and isolation

Every transaction holds a retry budget, statement deadline and conflict graph identifiers. Single-shard lock management and cross-shard wait dependencies must support cycle detection or a documented deadlock-prevention method. On abort, the engine MUST release all held resources and return a stable, machine-readable retry class. A client retry is permitted only when the protocol establishes no previously acknowledged committed outcome or when the write has an idempotency key recognized by the database. Serializable testing must include write skew, lost update, phantom, lock escalation, DDL interaction and a distributed dependency-cycle workload.

Acceptance tests: kill a coordinator before prepare, during votes, after outcome durability and during final participant notifications; repeat for simultaneous leader replacement and client disconnect. Verify that every transaction settles to exactly one decision, no participant exposes a partial committed result, and a replacement gateway can report the outcome deterministically.

22. Native storage internals and power-loss recovery

22.1 Page format and allocation lifecycle

The native row-store format MUST define a page header with magic, format version, object and page identity, page generation, page LSN, compression/encryption identifiers and authenticated integrity metadata. Data pages must encode slot directories or equivalent stable tuple locators, visibility/version pointers, free-space bookkeeping and bounded record lengths. Index pages require independently specified split/merge protocols, separator keys and root-generation publication. Freed pages cannot be reused while an older reader, snapshot, replica or recovery range could reference them.

How to implement: write the page codec and validator before the buffer pool. Use zero-copy access only after structural validation, overflow guards and checksum/authentication checks; fuzz corrupted and adversarial pages. Add stable reference fixtures for little- and big-endian simulation, nullable values, decimal precision, wide records and non-UTF-8 rejection. An allocator journal or recovery-safe page-map protocol must survive power loss between allocation and directory publication.

22.2 WAL sequencing, commit durability and checkpoint

Every persistent page update MUST be tied to a log sequence number and a declared redo/undo strategy. Before an updated page becomes durably visible, the WAL records needed to recover that page MUST have crossed the configured durability barrier. A transaction commit is acknowledged only after the authoritative commit decision is durably recorded under the selected durability mode. Windows and Unix file flush/rename semantics need platform-specific tests; a successful buffered write is not a durable commit.

How to implement: choose a physical/logical/hybrid WAL design through an ADR; implement deterministic redo and incomplete-transaction undo/discard semantics. Checkpoint manifests identify a consistent recovery starting point and validated log range, including catalog generations. Inject crashes between every log/page write, fsync, rename and checkpoint boundary. Recovery must be idempotent and operate safely if it crashes again halfway through replay. Protect against partial-sector and torn-page writes using an explicit technique such as double-write buffers, page images or verified copy-on-write pages.

22.3 Corruption modes, integrity and version GC

CRC or stronger authenticated page validation MUST detect accidental corruption; integrity protection for untrusted backup/media threats requires cryptographic authentication, not only checksums. Corrupt pages are quarantined with stable diagnostics; the server MUST NOT return silently substituted or fabricated values. Index rebuild is allowed only where authoritative rows and commits remain trustworthy. Snapshot version reclamation uses the minimum of active readers, backups, replicas and federation/export pins; stalled pins need operational visibility and bounded policy rather than arbitrary data deletion.

Acceptance tests: interrupted B+tree split, torn page, disk full during checkpoint, obsolete replica retention, redo retry, encrypted backup corruption, checksum mismatch and incomplete segment-compaction publication. Recovery must either reproduce an acknowledged state or explicitly refuse unsafe opening.

23. Security enforcement protocol and high-assurance architecture

23.1 Identity trust chain

A human user, application service, cluster node and federation partner MUST possess distinct identity classes. Authentication creates an immutable execution-security context containing principal ID, tenant ID, asserted and effective roles, DB/schema scope, authorization-policy revision, session binding, request ID, key/cert epoch and expiry. Administrative MFA is required through a supported identity provider or local secure enrollment; node and service communication requires explicit mutual authentication and certificate validation. Context is cryptographically bound or secured by a trusted internal channel, cannot be edited by a query fragment and is checked again at each trust boundary.

How to implement: centralize policy evaluation in one security kernel with a compiled policy AST; require every SQL operator, scan, cache lookup, connector, materialized view and internal RPC to carry a verified context. Add mandatory API-level authorization middleware but also enforce authorization at storage scans and result release: perimeter-only checking is insufficient. Use short-lived, scoped delegated tokens for workers and jobs; never forward administrator root credentials to compute workers.

23.2 Authorization equivalence, policy changes and side channels

The same protected table queried through a base scan, view, CTE, subquery, index-only plan, column store, cached result, remote fragment or data export MUST yield equivalent permitted rows and columns. Deny-by-default applies to unknown principals and unknown policy revisions. A policy-change event invalidates session permissions, compiled plans, cached results and queued outputs, according to a documented revocation window. Statistics, EXPLAIN, failure messages, audit metadata and timing behavior can expose sensitive information; the product MUST define redaction, minimum group-size policies for privacy-sensitive aggregates when configured, and restrictions on cross-tenant cardinality introspection.

How to implement: construct a negative-test matrix with identities × data classes × execution paths × operations, generate queries to force alternate plans and verify identical denials. Log policy decisions and policy revision hashes without dumping row payloads. Separate database owners, operational administrators, key custodians and audit readers; provide dual authorization for destructive key and backup operations where high-assurance profiles require it.

23.3 Cryptographic custody and incident containment

Use maintained and reviewed cryptographic libraries rather than inventing encryption. Enforce TLS for remote connections, mTLS for cluster RPC, authenticated encryption for rows/pages as appropriate and protected envelope encryption of WAL, spill, snapshots and backups. Define root/KEK/DEK hierarchy, tenant key scopes, rotation version, KMS/HSM outage behavior, independent recovery custody and

cryptographic erasure limitations. New cluster nodes enter quarantine until identity, version and policy checks pass; departed and compromised node certificates are revoked and any stale leadership is fenced.

How to implement: implement a cryptographic provider abstraction with explicit algorithm-suite IDs and format versions; test rotation on live replicas, failover mid-rotation, compromised backup keys, missing KMS access and restore with archived keys. Build an incident-isolation operation that prevents outgoing federation and new administrative sessions while retaining forensic audit. Cryptographic algorithm agility and key migration must not create a plaintext downgrade path.

23.4 Secure defaults, supply chain, independent evaluation

New instances bind administration to loopback, require first-admin enrollment, disallow universal default credentials and reject unencrypted remote administration. Build releases from pinned reviewed dependencies, generate SBOM and provenance, sign binaries/installers, scan code and dependencies and fuzz SQL, binary storage and network protocols. Security-sensitive modules require independent review, abuse-case tests and defined emergency patch timelines. Bradian Studio must use server-side authorization, secure browser session controls, safe SQL parameterization, rate limits and audit for privileged actions. A high-assurance production profile MUST fail configuration validation when audit sinks, transport encryption or root-key custody are unsafe.

Security gate: no release with unresolved critical authentication bypass, tenant escape, remotely exploitable memory corruption, unsafe default exposure, unverified restore or known WAL-data-loss scenario. Standards such as NIST, CIS and OWASP provide control frameworks, not certificates of compliance.

24. Cluster placement, elasticity and built-in routing controllers

24.1 Controller ownership and reliable control plane

The Cluster Fabric should provide membership, health, routing, placement, replica lifecycle and job controllers. Each controller operates on a versioned desired-state object and writes durable transitions that can be resumed after process loss. Only a fenced controller leader may publish new authoritative placement plans; watchers may propose but cannot independently move replicas. Control-plane metadata requires quorum semantics distinct from the data-plane shard quorum, with explicit bootstrap/recovery procedures.

How to implement: build controllers as modular reconciler loops sharing one typed cluster-state API, not ad hoc background threads. Membership admission verifies the cluster ID, node identity, supported protocol/storage versions and short-lived join authorization. Placement uses CPU/RAM, sustained I/O, storage headroom, hot-key pressure, failure domains, network locality, tenant policy and rebalancing bandwidth. Use hysteresis and cooldowns so short spikes do not provoke continuous shard movement.

24.2 Safe shard movement and rebalancing

A relocation is a state machine: planned → capacity-validated → destination admitted → snapshot copied → integrity-verified → log catch-up → quorum/ownership transition → routing-epoch published → old replica drained → cleanup. Each transition is durable and idempotent. During movement, stale gateways refresh shard ownership and cannot direct writes to a former leader. For strongly consistent shards the placement engine must never remove voters in a way that loses the required quorum or violates the configured number of independent failure domains.

How to implement: set per-node network/disk movement budgets; pause and resume migration under transaction pressure, key rotation or degraded replica health. Expose --plan/dry-run, predicted movement bytes, temporary capacity requirements, affected tenants and rollback limits. Provide a safety controller

that can reject unsafe operator changes even from the web UI unless a separately audited emergency process is authorized.

24.3 Connection gateways and client routing

Each gateway independently authenticates SQL clients, attaches stable session/transaction identifiers, identifies requested consistency and routes to the authoritative shard/worker. Read-only OLAP can use bounded-stale replicas only when the query contract permits; transaction-affined requests remain pinned to the proper coordinator for the statement or transaction. Connection balancing considers queue depth and saturation, not only round robin. On uncertain commit, drivers MUST NOT automatically replay side-effecting SQL without database idempotency guarantees.

How to implement: publish a signed or authenticated topology epoch and connection capability hints through a standard protocol; provide gateway-drain support and topology-aware drivers. Test gateway crashes during transaction prepare, streaming results, prepared statement use and authentication rotation. External VIP/DNS/load-balancer integration remains optional because no single in-process gateway should be a global SPOF.

24.4 Automatic capacity policy and honest scale promises

Bradian may move shards and balance work across already provisioned nodes autonomously. Adding physical servers requires an optional connector to an authorized cloud, VM or hardware provisioning environment; it is not implied by an internal scaling controller. Define autoscaling thresholds based on sustained saturation, spare capacity, replica safety and movement cost; do not claim every query becomes faster with more machines. Capacity plans must expose storage growth, compute headroom, failure-mode capacity and licensing/resource impacts.

25. Online schema evolution and catalog compatibility

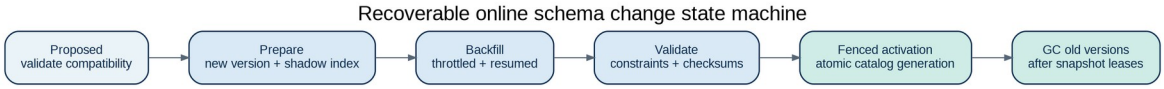


Figure 08: Recoverable schema evolution

Figure: Schema transitions use durable state and fenced activation; old versions retire only when safe.

25.1 Catalog generations and schema leases

Every database object has an immutable internal ID and monotonically versioned catalog generation. Active transactions and prepared statements resolve names to a catalog version; changes cannot reinterpret a column while a reader is using an older schema. Schema changes produce an auditable DDL job with separate proposed, prepared, backfill, validation, activation and cleanup phases. Old catalog generations remain available until all pinned transactions, query fragments and replicas have advanced beyond their last usage.

How to implement: enforce schema-version negotiation at coordinator and worker boundaries, implement lease or epoch fencing for DDL activation and protect incompatible client drivers through explicit error codes. Expose dependency graphs for foreign keys, indexes, views, materializations, triggers, published data products and permissions. Force invalidation and safe recompilation of dependent plans when object definitions change.

25.2 Backfill and constraint validation

For large-table changes, the engine creates a shadow representation or change-capture pipeline that can receive writes while historical rows are backfilled. Backfill is partitioned, bounded by I/O budgets,

resumable and indexed by durable progress markers. Constraints are validated against a coherent snapshot plus concurrent changes; partial validation cannot silently become authoritative. The final cutover requires a consistent catalog-generation switch and any necessary transaction fences.

Acceptance tests: kill any node during every DDL state, mix writes with an index build, add a foreign key under concurrent child inserts, change a column's collation, delay one replica's catalog update and run a prepared query pinned to the old schema. Every outcome must be safe, recoverable and visible in Bradian Studio.

26. SQL interoperability, correctness, types and client metadata

26.1 Published SQL compatibility profile

Bradian MUST publish a versioned SQL grammar and supported semantics matrix rather than promise unspecified full compatibility. The profile covers identifiers and quoting, reserved words, expressions, operator precedence, NULL and three-valued logic, transactions/isolation syntax, DDL/DML, CTEs, joins, window functions, collation, subqueries, RETURNING, UPSERT/MERGE, sequences and bulk import/export. Errors must expose machine-readable stable categories, SQLSTATE-equivalent codes and safe error details so drivers and applications can implement retries and migrations correctly.

How to implement: create a parser grammar with golden SQL fixtures; a typed binder and scalar interpreter act as a semantic reference. Differential-test supported queries against independently validated relational systems, preserving explicit documented differences rather than reproducing accidental behavior. Provide information_schema-style metadata views and versioned native catalog introspection. No compatibility branding should be applied to another database protocol until its wire and semantic compatibility suite passes.

26.2 Collations, temporal semantics and numerical precision

Text comparison, equality, ORDER BY and indexed uniqueness depend on collation and Unicode normalization choices. Each indexed text column stores a collation identifier and version; collation library upgrades trigger an assessment and potentially index rebuild. Temporal types require unambiguous instant/local-time distinctions, timezone database version recording, daylight-saving ambiguity handling and precise rounding. Decimal arithmetic must define overflow and rounding. Floating-point aggregate reproducibility must state deterministic-order guarantees and tolerance rules; SQL NULL must remain distinct from missing, zero and JSON null.

26.3 Drivers, cursors and session behavior

The native protocol specifies authentication negotiation, prepared execution, typed parameter/result mapping, transactions, cursors, paging, streaming, cancellation, idle behavior, error structure, TLS and topology refresh. JDBC/ODBC implementations need metadata APIs and catalog discovery, not only simple query execution. A pooled session must not leak role, tenant, temporary objects or prepared statements to the next borrower. Query cancellation cannot silently leave a transaction committed or abandoned in an unknown state.

Acceptance tests: JDBC metadata inspection, prepared-statement binary types, empty and NULL arrays, cursor resume/error behavior, backpressure, connection loss mid-stream, canceled DML, Unicode constraints and collations, concurrent schema change and driver retry classes.

27. Database migration, data integration and change delivery

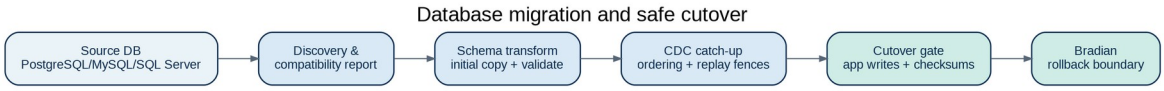


Figure 10: Safe source-to-Bradian migration

Figure: Migration is a planned, validated cutover, not a blind data copy.

27.1 Bradian Migration Manager

A standalone migration utility and Studio workflow should assess a source database, identify tables, keys, indexes, sequences, collations, views, stored routines and unsupported features, and produce a compatibility report before copying data. The initial copy must use a coherent source snapshot; optional ongoing change capture tracks source positions, replays data in order and verifies at-least-once transfer with idempotent target effects. Migration jobs record source/target schema revisions, row counts, ranges and checksums and can resume after interruption.

How to implement: begin with a read-only PostgreSQL schema-and-data adapter because its semantics and ecosystem are well documented, then add MySQL and SQL Server as separately verified adapters. Use a typed intermediate schema model and documented translation warnings. Treat unsupported triggers, functions, collations or transaction isolation differences as blockers or explicitly accepted limitations. The cutover workflow freezes or gates source writes, catches up CDC, validates correctness, switches client routing and records a rollback window.

27.2 Streaming contracts and ingestion isolation

Bulk imports specify batch atomicity, validation mode, duplicate semantics, error quarantine, type conversions, backpressure and restart checkpointing. CDC sources need source-identity tokens, ordering, schema-change markers, retention windows and data-deletion propagation. Connector credentials are scoped per source; connectors cannot inherit unrestricted database privileges. Columnar imports such as Parquet and Arrow IPC may be supported through reviewed parsers, with resource and decompression limits.

28. Durable operations, jobs, performance and automation

28.1 Persistent job scheduler

Maintenance and administrative actions must use a durable job framework with identity, privileges, state, dependencies, deadline, retry policy, progress, owner, audit and cancellation. Jobs include online index build, DDL migration, materialization refresh, statistics, backups, restore verification, compaction, integrity scrub, shard relocation, key rotation, data import and governed deletion. Jobs survive process restarts and must avoid duplicate unsafe effects when a worker crashes after performing work but before marking it complete.

How to implement: store job records in a transactional system catalog, claim work with lease/epoch fencing, write idempotent step checkpoints and emit structured events. Define PENDING, RUNNING, PAUSED, RETRY_WAIT, FAILED, COMPLETED and CANCELLED, plus reconciliation for orphaned tasks. Distinguish pause-safe operations from commit-critical operations that must complete or forward-recover; Studio should expose plan, impact, progress and operator approvals.

28.2 Resource reservations and starvation prevention

The resource governor must account for CPU threads, buffer-pool memory, query operator memory, spill disk, WAL bandwidth, network transfers and persistent storage. Query admission requires bounded

reservations so two fragments cannot each hold partial memory while waiting indefinitely for the other. OLTP reserves minimum resources while background maintenance and OLAP can be throttled. Distributed workload scheduling needs global fragment accounting and per-tenant limits, not merely a local process semaphore.

28.3 Workload intelligence and safe tuning

Collect query fingerprints, plan hashes, estimates/actuals, regression baselines and contention metrics under privacy-preserving retention. Automatic tuning may propose an index, statistics refresh, partition change or view but must produce a measurable plan with rollback and avoid silently changing query semantics. A workload-replay harness should reproduce mixed transaction and analytical pressure with source data anonymized or synthetic. Track performance, accuracy, resource usage and recovery together so a fast result with a wrong snapshot never counts as success.

29. Backup, PITR, ransomware resistance and global restore

29.1 Consistent distributed recoverable cut

A global backup MUST represent a set of shard/catalog/transaction-outcome states that forms a legal committed database history. Merely copying shard files at roughly the same time is insufficient. The design must specify either a global snapshot barrier with bounded quiescence or a verified consistent distributed snapshot protocol combined with WAL positions and global outcome records. Restore ordering ensures catalog, identities, transaction decisions and shard data agree at the selected recovery boundary.

How to implement: persist a backup manifest containing cluster ID, format versions, catalog generation, per-shard safe log positions, transaction outcome service checkpoint, encryption key versions, provenance and integrity hashes. Backups use authenticated encryption and independent credentials; the system supports immutable or write-once remote targets and offline copies. Every successful backup job must schedule or permit an isolated restore verification; restoration checks constraints, checksums and selected SQL reference queries.

29.2 Region loss, failover and safe rejoin

Disaster recovery plans distinguish single-node failure, shard quorum failure, regional isolation, full region loss, compromised cluster credentials and missing KMS. A new active region must have a clearly stated RPO/RTO and promotion/fencing authority. An old primary that reappears after isolation cannot resume write leadership without a new epoch and reconciliation/repair protocol. Driver topology updates, DNS/VIP cutover, audit continuity and backup ownership transfer are documented in runbooks.

Acceptance tests: restore to a chosen transaction boundary, simulate missing encryption keys, destroy a region while cross-shard transactions are in flight, attempt stale-primary writes, perform immutability/deletion attacks on backup credentials and verify key-custody separation.

30. Advanced analytical features and disciplined extension policy

30.1 Analytics SQL completeness

The column executor should eventually support rollup, cube, grouping sets, optimized window frames, approximate distinct estimates with reported error, quantiles, temporal bucketing and cohort analysis. Each algorithm has a scalar correctness oracle and explicit numeric precision contract. Result caching must include snapshot and authorization identities. Cross-shard partial aggregates must use mergeable sufficient states: sums and counts are straightforward under revision-aware contributions; distributed distinct persons and noninvertible measures require identities/sketches or controlled recomputation.

30.2 Specialized types as extensions

Time-series partition management, full-text inverted indexes, geospatial indexes and vector similarity search are separately versioned extension families. They MUST NOT delay WAL, ACID, SQL correctness and security qualification. When introduced, they use the same transaction, encryption, access policy, audit, backup and cluster-placement contracts as ordinary tables. Extension code must not bypass the storage kernel or arbitrarily load native code from SQL.

31. Formal v1.1 requirements and verification register

BRD-DTX-001 (P0) — Atomic and recoverable cross-shard transaction decision
Verification evidence: Coordinator/participant crash matrix
BRD-DTX-002 (P0) — Global snapshot ordering and documented safe-time model
Verification evidence: Clock skew and multi-shard history test
BRD-DTX-003 (P0) — Idempotent outcome lookup and retry classifications
Verification evidence: Duplicate/reconnect tests
BRD-STO-004 (P0) — WAL/page-LSN/checkpoint and torn-page protocol
Verification evidence: Power-loss fault harness
BRD-STO-005 (P0) — Crash-safe B+tree split, allocator and page reuse
Verification evidence: Indexed write/recovery replay
BRD-STO-006 (P0) — Verified format migration and corruption refusal
Verification evidence: Reader fuzz/compatibility corpus
BRD-SEC-008 (P0) — Verified context across SQL, RPC, caches and async jobs
Verification evidence: Authorization-path equivalence tests
BRD-SEC-009 (P0) — Safe diagnostics, inference and aggregate disclosure controls
Verification evidence: Metadata/plan leakage attacks
BRD-SEC-010 (P0) — Separation of privileged administrator duties
Verification evidence: Role/key/audit conflict matrix
BRD-SEC-011 (P0) — Revocation propagation and bounded enforcement delay
Verification evidence: Pending query/export revocation tests
BRD-SEC-012 (P0) — Release-blocking security controls and key rotation
Verification evidence: Pen test, threat model and rotation drills
BRD-CL-005 (P0) — Desired-state controllers, admission, epochs and fenced placement
Verification evidence: Split-brain / reconcilers fault tests
BRD-CL-006 (P0) — Safe online rebalance and replica lifecycle
Verification evidence: Shard-move failure matrix
BRD-CL-007 (P0) — Gateway session/transaction routing and safe retries
Verification evidence: Gateway interruption tests
BRD-DDL-001 (P0) — Recoverable multi-version online DDL state machine
Verification evidence: Concurrent write / DDL crash corpus
BRD-SQL-005 (P0) — Versioned SQL profile, metadata and stable error classes
Verification evidence: SQL differential and JDBC metadata
BRD-SQL-006 (P0) — Collation, temporal, decimal and NULL semantics
Verification evidence: Type/collation golden corpus
BRD-JOB-001 (P1) — Durable job state and idempotent resume
Verification evidence: Crash-at-each-step job tests
BRD-MIG-001 (P1) — Compatibility assessment and migration validation
Verification evidence: Source/target parity test
BRD-MIG-002 (P1) — Source CDC catch-up and recoverable cutover
Verification evidence: Interrupted migration and rollback
BRD-DR-001 (P0) — Cluster-consistent backup and PITR

Verification evidence: Cross-shard restore to legal cut
BRD-DR-002 (P1) — Immutable backup, isolated restore and safe regional promotion
Verification evidence: Ransomware and region-loss drills
BRD-OPS-007 (P1) — Global resource reservations and fair scheduling
Verification evidence: Mixed load starvation test
BRD-OPS-008 (P1) — Query regression and workload replay
Verification evidence: Reproducible seeded benchmark
BRD-EXT-002 (P2) — Safe specialized index/type extensions
Verification evidence: Contract, privilege and recovery test

32. Detailed implementation sequence and architecture decision gates

Gate 0 — Evidence, code audit, risk and specification closure

Audit the repository, current source and prior DER analytics artifacts. Produce an implementation inventory with actual passing tests, an explicit unsupported-feature list, a traceability matrix, security threat model, ADR backlog and storage/protocol compatibility map. Prototype critical uncertainties (fsync correctness, MVCC/GC, global transaction ordering and authorization propagation) in controlled experiments before committing to persistent formats.

Gate 1 — Portable kernel and correct single-node transaction

Implement a native process, bounded memory/runtime layer, page and WAL codecs, catalog, B+tree, one-table SQL vertical slice, transaction commit, rollback, checksums and restart recovery. Require crash injection at durable-write boundaries, reference SQL results, clean installer and secure local identity. Do not advance with an in-memory-only mock.

Gate 2 — Relational compatibility and safe schema lifecycle

Add compound keys, foreign keys, constraints, prepared statements, transactional DDL behavior, online schema jobs, catalog generations, stable SQL error categories and JDBC metadata. Require concurrent constraints/DDL tests, local security-path equivalence and differential SQL conformance.

Gate 3 — Columnar engine and commit-consistent HTAP

Retain the original native column format, vectorized operators and CBO. Build committed journal projection with clear watermarks, retractions, materialization lineage and strong/bounded-stale snapshots. Test a mixed OLTP/OLAP workload against an oracle at identical visible database revisions.

Gate 4 — Single-cluster replication and safe placement

Implement node identity and mTLS enrollment, consensus-replicated shards, leader fencing, native gateways and controlled replica movement. Require safe quorum loss behavior, failover and rebalancing crash tests before attempting multi-shard transactions.

Gate 5 — Distributed commit and global snapshot correctness

Implement cross-shard atomic commit, coordinator recovery, timestamp/safe-time protocol and serializability tests. Require adversarial network faults, clock skew and deterministic outcome resolution. Cross-shard consistency is a standalone gate and cannot be declared complete because local ACID passes.

Gate 6 — Studio, durable operations and migration

Deliver browser Studio backed by the same privileges as SQL/API paths; durable jobs for DDL, backup, compaction and rebalancing; migration assessment, initial copy and validation. Require accessibility and browser security testing, API/driver conformance, and job restart idempotency.

Gate 7 — Federation and site autonomy

Implement separate installation identities, authorized delta contracts, schema-version checks, idempotent change replay, explicit conflicts and coverage-aware federated analytics. Validate extended WAN outages and do not equate offline autonomy with global strong consistency.

Gate 8 — Cluster-consistent recovery and hardening

Complete global backup/PITR, immutable backups, KMS recovery, key rollover, region-loss procedures, signed installers, scanning, fuzzing, attack simulations and independent security review. Release cannot proceed with unverified disaster restore or high-severity tenant escape.

Gate 9 — Qualification and external reproducibility

Run published SQL/HTAP/distributed correctness suites; cold/warm and mixed-load benchmarks on declared hardware; platform-specific installer and crash-recovery tests; operator runbooks; security documentation; upgrade/rollback drills and conformance reports. Performance claims must include resource costs, topology, cache state and measured correctness.

33. Acceptance scenario catalogue and test harness design

33.1 Scenario families and invariant assertions

Transaction history: generate serializable and snapshot-isolation workloads with controlled interleaving, duplicate requests, retries, stale followers and leader loss. Record read/write sets and commit outcomes; analyze resulting histories for the claimed isolation model. Validate uniqueness, foreign-key, check constraints and consistent read snapshots across all participating shards.

Storage fault injection: inject crashes after each critical WAL append, flush, page write, manifest rename, checkpoint record and compaction transition. Run restart repeatedly to detect non-idempotent recovery. Compare logical rows, indexes, aggregate projections and catalog dependencies to a verified reference state; silently repaired corruption is a test failure unless recovery policy explicitly permits rebuilding a known derivative.

Security: combine principal, tenant, role, SQL feature, execution path, node placement and policy revision. Force index-only scans, cached materializations, distributed joins, temporary spills, export workers and backup preview paths. A denied access MUST remain denied under every plan, and audit must identify the event without exposing protected row values.

Operations: lose a gateway, replace a replica, stall a placement-controller leader, pause DDL during backfill, rotate keys with an unavailable KMS, restore a backup into a new cluster and simulate total loss of a region. Verify operator-visible warnings, no invented data completeness and bounded repair.

Portability: verify signed package install/uninstall on the published OS matrix; identical on-disk format fixtures; filesystem permission and path behavior; Unicode/collation stability; platform-dependent flush semantics; and successful backup portability. Cross-OS mixed clusters are conditional on explicit qualification, not merely shared source code.

33.2 Measurable release evidence

Every requirement in Sections 16 and 31 has an owner, code reference, automated test ID, threat scenario where applicable, benchmark or operator drill, and release status. Store test seeds, failing interleavings, fuzz corpora, checksum manifests and benchmark scripts as reproducible artifacts. Run nightly differential/fault suites and a separate release qualification matrix. User-facing release notes MUST identify implemented SQL grammar, supported storage versions, deployment modes, secure defaults and known limitations.

34. Architecture decision record catalogue

ADR-001 — Authoritative row-store page and B+tree design
ADR-002 — WAL logging mode, torn-page protection and checkpoint algorithm
ADR-003 — MVCC chains, lock model, isolation and vacuum horizon
ADR-004 — SQL grammar, metadata protocol and error-class model
ADR-005 — Global timestamp / safe-time and snapshot model
ADR-006 — Distributed 2PC-style outcome recording and recovery
ADR-007 — Shard consensus, leader leases, epochs and membership changes
ADR-008 — Column projection format and strong-snapshot delta merge proof
ADR-009 — Online schema-generation activation and recovery
ADR-010 — Security context, role inheritance and policy enforcement points
ADR-011 — Key hierarchy, KMS boundary, algorithms and rotation
ADR-012 — Global backup recovery cut, PITR and cross-region promotion
ADR-013 — Placement controller, movement budgets and autoscale policy
ADR-014 — Native SQL driver protocol and compatibility testing
ADR-015 — Safe extension process boundary and connector capabilities

35. Architecture diagrams index and document status

The figures embedded in Part I and Part II describe: (1) system decomposition, (2) distributed transaction sequence, (3) WAL and storage recovery, (4) HTAP consistency, (5) cluster/gateway routing, (6) federated installations, (7) security-context propagation, (8) online DDL, (9) backup and recovery, and (10) database migration. They must be maintained alongside their ADR and tests when the architecture changes. Diagram nodes depict logical responsibilities, not mandatory independently deployed services.

Status: Bradian Master Architecture & SRS v1.1 is a target design and requirements baseline. It is deliberately explicit about implementation strategies and test evidence while distinguishing proposed mechanisms from approved ADRs and working code. Product security, SQL conformance, fault tolerance and performance claims require proof by implementation and external review. Nothing in this document certifies those qualities before testing.